Soft Comput (2016) 20:1763–1779
DOI 10.1007/s00500-015-1705-6

FOCUS

TACIoT: multidimensional trust-aware access control system for the Internet of Things

Jorge Bernal Bernabe[1] · **Jose Luis Hernandez Ramos**[1] · **Antonio F. Skarmeta Gomez**[1]

Published online: 15 May 2015

Abstract Internet of Things environments are comprised of heterogeneous devices that are continuously exchanging information and being accessed ubiquitously through lossy networks. This drives the need of a flexible, lightweight and adaptive access control mechanism to cope with the pervasive nature of such global ecosystem, ensuring, at the same time, reliable communications between trusted devices. To fill this gap, this paper proposes a flexible trust-aware access control system for IoT (TACIoT), which provides an end-to-end and reliable security mechanism for IoT devices, based on a lightweight authorization mechanism and a novel trust modelthat has been specially devised for IoT environments. TACIoT extends traditional access control systems by taking into account trust values which are based on reputation, quality of service, security considerations and devices' social relationships. TACIoT has been implemented and evaluated successfully in a real testbed for constrained and non-constrained IoT devices.

Keywords Trust model · Access control · Internet of Things · Security framework

Communicated by A. Jara, M. R. Ogiela, I. You and F.-Y. Leu.

✉ Jorge Bernal Bernabe
jorgebernal@um.es

Jose Luis Hernandez Ramos
jluis.hernandez@um.es

Antonio F. Skarmeta Gomez
skarmeta@um.es

[1] Department of Information and Communications Engineering, University of Murcia, Murcia, Spain

1 Introduction

The Internet of Things (IoT) (Atzori et al. 2010) represents the evolution from an enterprise-centric Internet to a smart object-centric paradigm, in which physical objects are being enabled with ambient intelligence and network abilities allowing a constant data exchange. This information is aggregated, processed and used to provide pervasive and innovative services, transforming our surrounding environment into an intelligent and ubiquitous space (Weiser 1991). This trend is being accelerated by significant advances in wireless communication technologies and being exploited to realize new scenarios such as the next generation of smart cities (Schaffers et al. 2011).

The deployment of pervasive systems contributes significantly to the provisioning of smart services to improve the society's quality of life. However, in this new digital scene, physical objects around us are vulnerable to attacks and abuse, and ensuring a secure communication between heterogeneous and reliable devices, is essential to achieve the acceptance of all IoT stakeholders. Consequently, the application of security mechanisms has to deal with the high degree of heterogeneity and dynamism of smart objects making up such ubiquitous environments.

The design of trust and reputation models, as well as its application to access control mechanisms, has been traditionally focused on the deployment of security solutions for distributed systems. However, they have been usually tailored to wireless sensor networks scenarios, without considering the inherent requirements and features of IoT scenarios. In particular, these environments require lightweight, flexible and adaptive access control mechanisms to cope with the pervasive nature of such global ecosystem, where heterogeneous devices are accessed ubiquitously. In addition, to get an accurate trust assessment to be taken into account during

the access control, it is also needed to consider the vagueness associated with the information that is detected by the devices in such environments.

In this direction, this work proposes a trust-aware access control mechanism for IoT (TACIoT). To deal with the vagueness associated with contextual information in pervasive scenarios, the paper proposes a novel trust modelbased on fuzzy logic, which is the TACIoT cornerstone. Such model follows a multidimensional approach to enable an accurate trust value computation for IoT devices. In particular, unlike previous models that usually only consider reputation and feedback, it uses security aspects and social factors between IoT devices, which are then used by smart objects to drive their access control logic. TACIoT is framed in our IoT security framework (Bernal Bernab et al. 2014), which is based on the Architectural Reference Model (Bassi et al. 2013) (ARM) from IoT-A EU project. The security framework is intended to provide a holistic security approach to be deployed and instantiated over emerging IoT scenarios. In addition, TACIoT has been implemented for both constrained and non-contrained devices and tested in real scenarios. A set of evaluation results are analyzed and discussed, to demonstrate the feasibility of our flexible and lightweight trust-aware access control system for IoT.

The remainder of this paper is structured as follows. Section 2 provides a description of some related proposals regarding access control and trust models for IoT scenarios. Then, Sect. 3 describes the security framework in which the application of TACIoT is considered. Section 4 provides an overview of the main foundations of the access control mechanism employed in TACIoT. Afterwards, a detailed description of the proposed trust model is given in Sect. 5. Section 6 delves into the application of the the TACIoT in IoT. In Sect. 7, a set of experimental results of the proposal are shown, and in Sect. 8, the paper ends up with some conclusions and an outlook of our future work in this area.

2 Related work

The realization of IoT scenarios imposes significant security and privacy restrictions, since everyday physical objects are being seamlessly integrated into the Internet infrastructure, becoming part of our digital sphere. Therefore, the enormous potential of IoT can be threatened if the security and privacy considerations are not properly tackled (Heer et al. 2011; Medaglia and Serbanati 2010). In this direction, several efforts are beginning to be developed to achieve a full acceptance of IoT stakeholders, and consequently, a broader adoption of this emerging paradigm.

In this context, the application of consolidated authorization models such as Role-based Access Control (RBAC) (Ferraiolo et al. 1995) or Attribute-based Access Control (ABAC) (Yuan and Tong 2005) presents significant drawbacks due to the complex management of authorization policies, which is usually required. This has led to the design of solutions in which smart objects are agnostic of the security mechanisms and access control models. This problem has been identified by recent proposals that suggest the use of authorization tokens (Gusmeroli et al. 2013; Mahalle et al. 2012) as an access control mechanism for IoT scenarios. Under the main considerations of these works, Distributed Capability-Based Access Control (DCapBAC) (Hernández-Ramos et al. 2014) was recently introduced as a feasible access control approach for IoT, even in scenarios with constrained devices and networks. While authorization decisions can be still externalized, DCapBAC allows a distributed approach in which smart objects are enabled with access control logic, and a central entity is not required during the communication with IoT devices.

However, DCapBAC does not provide trust mechanisms that take into account the trustworthiness of the involved IoT devices to make access control decision accordingly. In this direction, Chen et al. (2011) proposes a trust management model based on QoS parameters such as package delivery ratio and end-to-end packet forwarding ratio. It uses fuzzy sets to manage trust and reputation but provides a set of results based on NS-3 simulations. Authors in Ben Saied et al. (2013) propose a context-aware and multi-service trust management system that is intended to fit the inherent requirements of IoT scenarios. This mechanism uses different metrics to assess the trustworthiness of devices with different hardware capabilities, to cope with the heterogeneous nature of IoT. Moreover, the use of social properties has been employed in Nitti et al. (2013) to assess the degree of trustworthiness between IoT devices. The proposed approach is based on the Social IoT (SioT) paradigm (Atzori et al. 2012), in which nodes can autonomously establish social relationships with each other. They present two models for trust management; in the subjective model, each node calculates the trust of its friends based on its own experience and the opinions of friends, while in the objective model, information is distributed so that all devices use the same information. Furthermore, a set of simulations are provided to evaluate the model in the presence of malicious nodes. Following this approach, Bao and Chen (2012), Bao et al. (2013) propose a dynamic trust model which takes into account metrics related to social cooperativeness, community interest and recommendations. As previous proposals, a set of theoretical results and simulations are provided. The integration of trust models in an access control mechanism for IoT is considered in Chen et al. (2012), which proposes a reputation-based RBAC access control model (R2BAC). This model uses only QoS metrics to calculate trust values, which are assigned to roles. Moreover, authors in Mahalle et al. (2013) propose FTBAC, a trust-based access control solution using the

linguistic values of experience, knowledge and recommendation as inputs. Then, these fuzzy trust values are mapped to access privileges. Although scalability and energy consumption are simulated, a real implementation of the solution is not provided.

The previous trust models only consider a common small set of parameters to evaluate the trustworthiness of IoT devices (like QoS or reputation). In contrast, the model proposed in this work considers most of these parameters altogether, adding even a new dimension of trust regarding security. This security dimension allows considering security evidences, which are inferred from security mechanisms being employed in a specific transaction, as part of the trust model which quantifies the trustworthiness. Furthermore, to cope with the vagueness of information in pervasive environments, the model uses fuzzy logic to infer the trust values. In addition, our trust model has been integrated into a lightweight and flexible access control mechanism based on DCapBAC (Hernández-Ramos et al. 2014), with the aim of making IoT devices aware of other devices' trust scores and drive their access control decisions accordingly.

3 IoT security framework

The application of security mechanisms over the IoT paradigm needs to face different challenges because of the heterogeneous nature of devices and networks comprising this global ecosystem. In recent years, several European initiatives have defined different architectures to design IoT services and applications under a common view. Typically, these approaches were tailored to specific domains addressing small subset of requirements regardless of the global nature of IoT. This was identified as one of the main barriers for a broad adoption of IoT. In this direction, the EU FP7 IoT-A project represents the most remarkable initiative for the creation of a harmonized vision of IoT in Europe, by optimizing the interoperability among isolated IoT applications to create a global ecosystem of services under a common understanding. The main result of IoT-A was the definition of an *Architectural Reference Model* (ARM) (Bassi et al. 2013) for IoT systems, to promote a common understanding at a high abstraction level, through the description of essential building blocks.

The results from IoT-A promoted the emergence of additional initiatives adopting ARM as the starting point of design activities, such as the EU FP7 IoT6 (Ziegler et al. 2013) or BUTLER (Architecture 2013) projects. However, a common feature of the resulting architectures of these efforts is that they are not focused on the definition of suitable security and privacy mechanisms for IoT scenarios, supporting aspects such as privacy by design (Langheinrich 2001), and data minimization principles. These requirements cannot be tackled at later stages during the development of IoT services, but they must be taken into account during the design to build innovative, secure and privacy-preserving IoT applications. To address this issue, we have designed an ARM-compliant security framework, whose description can be found in Bernal Bernab et al. (2014). This framework, which is shown in Fig. 1, proposes an extension of the security functional components of ARM enabling more flexible sharing models, in which physical context information

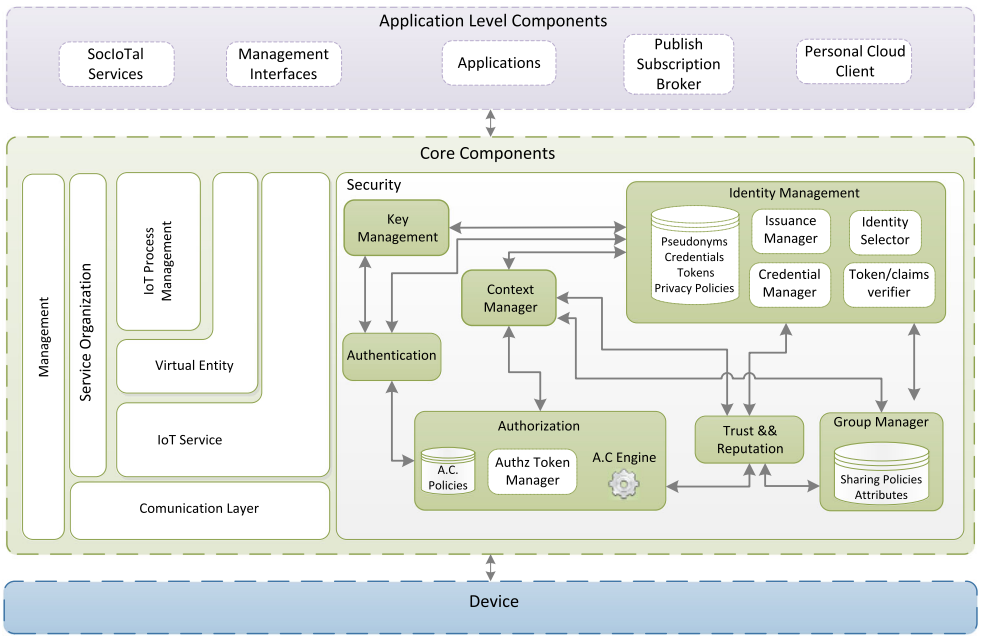

Fig. 1 ARM-based security framework for the IoT

is considered as a first-class component to drive the security behavior of IoT services.

The IoT framework makes use of security policies to drive the behavior of the different components. The *Context Manager* is in charge of maintaining, in real time, the context taken from localization enablers and monitoring processes. This component is of paramount importance in our framework since other components such as the *Authorization and Trust&&Reputation* Components make use of the context in their policies. The former makes authorization decisions taking into account context such as time, location or device status (e.g., battery level). The latter can access the *Context Manager* to obtain evidences (e.g., security aspects) about the peer device being evaluated to drive the trust computation process. Even the *Identity Manager* component is driven by security policies, taking into account the context and privacy concerns to change dynamically the user's partial identities used against services to ensure privacy.

The security framework is currently being evolved and implemented under the EU FP7 SocIoTal project (A socially aware citizen-centric 2013), the work presented in this paper addresses some of the main components and interactions involved. Namely, this paper delves into the Trust Manager component proposing a dynamic and flexible trust and reputation model based on fuzzy logic. The model takes into account reputation, quality of service, social relationships considerations as well as security aspects. This model has been implemented by the Trust Manager and then integrated within the Authorization System to enable a secure information exchange between trustworthy IoT services. Thus, the Authorization System component can make authorization decisions based on devices' trust scores. A detailed description of both the Authorization System and the trust modelis provided in the following sections.

4 Flexible and lightweight access control for IoT

The seamless integration of physical objects into the Internet infrastructure is fostering the design and development of new security mechanisms to be deployed on the resulting IoT scenarios. Indeed, standard security solutions were not designed taking into account the heterogeneous nature of this IP-based global ecosystem, and consequently, a deep revision and adaptation of these mechanisms need to be considered. To enable an end-to-end transparent information exchange between IoT devices, the Constrained Application Protocol (CoAP) (Shelby et al. 2014) has been recently announced as the standard transfer protocol at the application layer to be deployed on IoT scenarios. While CoAP is based on the same RESTful principles as HTTP, it enables the realization of embedded services even on constrained devices and networks. Although CoAP provides several modes for securing the protocol through a binding to the Datagram Transport Layer Security (DTLS) (Rescola and Modadugu 2006), it does not address the application of authorization and access control mechanisms for IoT. These challenges have received a huge attention from research community and recently several efforts are starting to emerge in this direction. However, due to the severe resource constraints of IoT devices, most of these proposals assume the use of an online central entity, which is responsible for security and access control mechanisms. Therefore, these approaches require end devices to maintain a connection to the central entity when they receive an access request, which may not be possible in *Low-Power and Lossy Networks* (LLNs). Moreover, the use of this central component makes end devices agnostic of any security mechanism preventing end-to-end security to be achieved. In addition, these solutions require a greater exchange of messages for end devices, which can be a critical factor in resource-constrained devices and networks.

Due to the lack of a suitable IoT access control solution, the IETF Authentication and Authorization for Constrained Environments (ACE) WG has been recently established to produce standardized authentication and authorization mechanisms to be deployed on devices and networks with tight resource constraints. Currently, the ACE WG is focused on the definition of requirements and considerations that must be addressed by security mechanisms that are designed for IoT-constrained environments (Seitz and Selander 2014). It also defines an initial architecture with different requirements and actors (Gerdes 2014). Under the set of initial considerations of the ACE WG, the access control based on capability tokens as been postulated as a feasible access control approach to be deployed on IoT-constrained environments.

The access control system adopted in this paper is based on the author's previous paper entitled Distributed Capability-Based Access Control (DCapBAC) (Hernández-Ramos et al. 2014). DCapBAC allows a distributed approach in which constrained devices are enabled with access control logic by considering suitable technologies and mechanisms for IoT environments. The capability-based access control system is being deployed and tested in SOCIoTal scenarios as key part of the Authorization Systemof security framework. The main DCapBAC features are:

- It is based on authorization tokens which are represented with JavaScript Object Notation (JSON) (Crockford 2006) instead of XML-based representations which are not recommended for IoT. The tokens are comprised of different fields, among them: issuer, subject, signature, and the access rights (action, target resource and conditions).
- It uses CoAP (Shelby et al. 2014) as lightweight application layer protocol to make access requests, in which the token is attached.

– It defines an optimized Elliptic Curve Cryptography (ECC) (Marin et al. 2013) library for providing end-to-end security via digital signature. ECC, unlike other cryptographic schemes, requires lower computing and memory requirements as well as smaller keys.

DCapBAC focuses on the authorization enforcement stage, whereby the involved devices can be mutually authenticated and the capability tokens are exchanged securely and then validated for a specific access request. In addition to such a proposal, this paper addresses the access control procedure from the beginning, when the capability tokens need to be generated. To this aim, the Authorization Manager is endowed with a Policy Decision Point (PDP), which is able to make authorization decisions using a policy engine. A Token Manager subcomponent generates the tokens based on the PDP decisions. The policies are defined using the Extensible Access Control Markup Language (XACML) (Rissanen 2012).

XACML supports Attribute-Based Access Control (ABAC) approach where access control policies can employ directly subject properties (e.g., age, location, roles, etc.), as well as resources and environmental properties. Thanks to the usage of ABAC, unlike in traditional RBAC models, the number of rules can be reduced, at the expense of more powerful (and complex) rules and more processing and data availability requirements. Nonetheless, although XACML requires moderate computation capabilities to run the engine and process the policies, it is not actually a problem because the PDP is not supposed to be deployed in constrained devices, which only require enough computation resources to validate the generated tokens. XACML also allows to describe obligations in the policies, which enables the PDP to define directives that must be carried out by the Policy Enforcement Point (PEP) before an access is approved. This feature is important in TACIoT proposal since it is used as a basic mechanism to indicate to the target devices acting as PEP, that they must allow access to requester devices as long as they satisfy the trust values.

Thus, our Authorization System allows target devices to take into account trust values about subjects devices before allowing them to access its resources. During the token validation process done in the target device, the Authorization Systemdeployed in the device only checks that whether the trustworthy values about the subject or requester are not below a specific threshold value, otherwise it does not permit the incoming request. The trust values about devices are computed by the Trust Manager component of the framework, that can be deployed in the target device for non-constrained devices, or in another entity in case of the device has not enough hardware resources (both cases are addressed in this paper). The Trust Manager implements our trust model specially meant for IoT, which is based on fuzzy logic and it is described in the following section.

It should be noticed that each component in the framework can assume different functionalities depending on the entity in which it is deployed. The Token Manager entity runs a special Authz Manager component of the framework with issuance functionalities. The token issuance (where the authorization decision is made based on the XACML policies) is carried out just once by the Authz Manager component (unless there was a revocation or the token had expired). This entity is deployed outside devices so that devices do not need to store access control policies, which could not be computationally feasible in many IoT scenarios. This approach allows subject devices to access target objects as many times as required without running the token issuance process. Thus, the subject's request only needs the capability token to be validated by the target, checking that the signature is valid and the trust value is over the threshold. It avoids evaluating the access control policies for each access request.

5 Trust model for the Internet of Things

The trust model follows a multidimensional approach to calculate the overall trustworthiness about an IoT device. It considers different properties that could be taken into account in the Internet of Things paradigm. These properties are considered to come up with an overall value about four main trust dimensions. Thus, in addition to traditional considerations such as service feedback and reputation, our trust model takes into account security aspects and social relationships within the peer device. In the end, this approach leads to a more accurate and reliable value of trustworthiness about a given IoT device.

5.1 Multidimensional trust model

The trust model follows a hierarchical approach in which the different dimensions are split into categories and subcategories, which in turn are composed by measurable properties. This hierarchical approach enables the trust model to be extensible, allowing users to consider and include new properties to the model. Nonetheless, the trust quantification procedure is the same regardless of the amount of properties taken into account. In fact, some of the trust properties explained below could be optional in case the Trust Manager could not have means to obtain evidences about such a property. This would depend on computation capabilities of the device where the Trust Manager is deployed as well as the implementation of the ContextManager component which provides evidences about many of the properties. The Trust Manager is deployed in the same target device when it comes to non-constrained devices like in an smartphones, whereas

it can be deployed outside in case of constrained devices. Thus depending on the use case and the specific devices, some of the properties used to quantify trustworthiness can be considered or not.

Each trust property is measured differently according to its nature. There are basically two kinds of properties, those properties measured as a real number value $x \in [0 \ldots 1]$ e.g., service availability, and those measured as a booleans, i.e., whether a device features a certain expected security aspect or not. In this last case, x is a member of the binary set $X = \{0, 1\}$.

It should be noticed that property values must be normalized into positive values in range [0,1] using the equation: $e = \frac{a - a_{\min}}{a_{\max} - a_{\min}}$ where $a_{\min}$ and $a_{\max}$ refers to the minimum and maximum expected values for a given type of property.

The following subsections describe the four main dimensions and the trust properties covered in each of them.

5.1.1 Quality of service dimension

This dimension refers to the evaluation of the overall quality of service provided by a device. It is done recapping evidences of previous interactions within the peer device being analyzed. The dimension is, in turn, comprised of four main indicators or properties that help to come up with an accurate overall value of quality of service. All of these properties are real numbers in the $[0 \ldots 1]$ interval.

The *%Successful-Interactions* property is the most important property in this dimension since it recaps the percentage of successful interactions over the total amount of previous interactions within the device.

In addition, the *Availability* property indicates the proportion of time that the IoT device is operating. It is measured as the ratio of the total time that the IoT device can be used in a given time interval.

The network *Throughput* when accessing to the IoT service can be also considered to work out the overall quality of service of a given device. The throughput is measured in bits per seconds and is defined as the rate of successful packets delivery over a communication channel. It can be affected by the physical medium or the device processing power.

Similarly, the average network *Delay* within the device measures the overall quality of service. It indicates how long it takes for a bit of data to travel across the network from one device to another.

5.1.2 Security dimension

As it was explained in Sect. 3, the Trust Manager can access the current context by means of the Context Manager component. In this sense, the Context Manager is able to generate dynamically, and in real time, security evidences about the current security mechanisms employed during the actual transaction between two IoT devices. This security information is employed by the Trust Manager in the moment of computing trust. Then, these trust values drive the access control, and therefore, can avoid the transaction before being carried out. The security dimension is set up by the aggregation of different security categories and its properties are explained below.

The *AuthN-AuthZ-System* category refers to the different authorization and authentication mechanisms that are supported by the device in the moment of performing the request. It is in turn split into the mechanisms that are available for different layers, where each mechanism is a boolean property that can be considered by the trust model. In the Network layer, mechanisms such as EAP-IKE, EAP-SIM, EAP-AKA can be employed in the IoT world. Regarding authorization and authentication mechanisms for IoT at the application layer standards like OAuth, SAML and OpenID can be adopted.

The *Communication-Protocol* security category models evidences about the communication protocols which are used during a transaction between two devices to provide confidentiality and integrity in the communication. The communication protocol employed has a direct impact on the reliability of the peer device. This category is also split into two subcategories, that is, network layer protocols like IPSec, and transport layer protocols like TLS or DTLS.

The *Network Scope* category evaluates the requester trustworthiness and benevolence from the point of view of its network prefix. Thus, devices in the same local network or with the same network prefix are usually more secure than those devices belonging to global or external networks. Thus, our trust modeltakes into account, to certain extent (given by the configured weights), the network scope property to evaluate the trustworthiness of the peer device.

The *Device-Intelligent-Capacity* property denotes the amount of risk assumed by the fact of interacting with a powerful device with high computing capabilities. As the device is more capable, it has more probabilities to act dangerously and therefore the risk assumed is higher. The device can identify the peer device and classify it in three levels according to its power and capabilities. Namely, constrained devices (e.g., sensors), common IoT devices (e.g., TVs, smart phones, smart watches) and traditional and powerful machines (e.g., laptops, servers...).

5.1.3 Reputation dimension

The trust modeltakes into account recommendations from other devices about a particular device j. Let O_j^i be the Opinion about device j given by device i. The trust modelweights each recommender to limit their influence according to a recommender's behavior in the past. Thus, the opinions are subject to a credibility process where each reputation

evidence coming from a device i is subject to credibility factor Cr_i in the interval $[0 \ldots 1]$, where 1 represents the highest credibility. Therefore, the Reputation property in our trust modelis given by $R_j^i = O_j^i * Cr_i$.

In addition, since usually recommender's honesty is not ensured, recommendations are subject to a filtering process to reduce the impact of deceitful recommendations. Recommendations are considered as long as they are similar to an entity's direct experience. The difference between the expectation value obtained based on direct experience and the expectation value obtained by the recommendation is calculated. Then, the recommendation is considered just in the case the difference is less than a predefined threshold.

Each device i stores the direct evidences and recommendations provided by other devices to quantify trust of each peer device j. Nonetheless, it should be noticed that in the IoT world, constrained devices could not be able to cope with large amount of historical evidences about devices to compute trust. Indeed, these kind of devices can be configured to drop evidences about devices which they do not interact for a long period of time.

Notice that the quantification of the reputation property (like some others properties described in this section) requires a minimum hardware capabilities, which means that it could be not feasible to take into account this property by those Trust Managers deployed in constrained devices. Thus, this trust property is envisaged to be mainly handled by a Trust Managers deployed in non-constrained devices.

5.1.4 Social relationship dimension

The proposed trust model also considers social parameters as part of the trust quantification for a specific IoT device. These metrics are based on the emerging Social IoT (SioT) paradigm, in which IoT devices are able to establish social relationships with each other. In this case, we consider smart objects can be grouped into different trust bubbles or communities according to the social relationships which are set among them. For example, a *Personal bubble* B_p is a set of smart objects that are in contact because they belong to the same owner. Similarly, the smart objects can be connected each other to set up *Family Bubbles* B_f, or *Office Bubble* B_o, depending on the kind of the social relationship among them. Other kinds of relationships are *Parental object relationship* and *Co-location object relationship*. The former is defined among similar objects, built in the same period by the same manufacturer. The latter is established among objects that are placed in close locations but without needing to be placed always in the same places, i.e., among objects whose distance in a certain moment is lower than one predefined threshold. Beyond the concept of *Bubble*, a *Community C* is a set of smart objects that are in contact because they share a set of common interests.

It usually happens that, as the social relationships between devices get closer, the trust relationship among them is stronger. Our trust modeltakes into account the kind of social relationship between the evaluated device j when assessing the trust of such a device. In this sense, the weights given to a kind of relationship between the devices are different according to the links established among them. For instance, if the device j being analyzed belongs to the device i own Personal Bubble B_p, the social relationship will be higher when it only belongs to Family-Bubble B_f. In general, the weights assigned by the trust modelto the social relationships are configurable by the user in the interval $[0 \ldots 1]$ and should satisfy $w_{B_p} > w_{B_f} > w_{B_o} > w_C$.

In addition, in case the devices i, j do not belong to the same community or bubble, it can be evaluated the degree of *Interest-In-Common* between the two devices as well as the *Friends-In-Common*. The Interest-In-Common I_j^i trust property can be calculated as the ratio between the interest that both devices share over the total amount of interests of the evaluator device $I_j^i = \frac{\text{interests}(i) \cap \text{interests}(j)}{\text{interests}(i)}$. Similarly, the Friends-In-Common F_i^j property of the trust modelis calculated as the ratio between the number of friends that both devices have in common, and the total amount of friends of the evaluator device $F_j^i = \frac{\text{friends}(i) \cap \text{friends}(j)}{\text{friends}(i)}$

It should be noticed that to quantify the interests and friends-in-common the devices should be able to exchange, in a common way, their list of interests (e.g., services and capabilities) as well as the lists of friends.

5.2 Trust dimensions quantification

Each device is able to provide a set of services $\text{Services} = \{s_1, \ldots, s_z, \ldots s_m\}$, each one providing a certain functionality or resource. The trust modelfeatures the *Service-Importance-Factor* $S_{s_z}^j$, which aims to give different degrees of importance to each service provided by the device j. Thus, relevance interactions can be given more importance to the overall trust value about a device j, whereas irrelevant ones can be discriminated. This factor helps to avoid interactions with malicious nodes that act properly when providing certain minor services, but wrongly when providing the important ones. The $S_j^{s_z}$ takes values in the range of $(0 \ldots 1]$, where 1 means highly relevant service and 0 irrelevant service. The Trust Manager holds evidences about each property p of the model and weights them by the *Service-Importance-Factor*. Thus, the evidence value x about a trust property p regarding a device j, is given by $x_j^p = v_j^p * S_{s_z}^j$, where v_j^p is the value obtained for the property p.

To calculate the expected trust value about an IoT device j, our model recaps the evidences about the different trust properties explained in Sect. 5.1. Then, when an authorization decision needs to be made, and therefore it is necessary

to compute trust, the latest evidence along with the past ones is taken into account to have a more reliable value.

In the trust model, the average rating $\overline{a}$ for each trust property p and peer IoT device j is calculated with a weighted arithmetic mean. The mean is weighted with the evidences of particular past interactions n. It ensures that values obtained from the newest interactions contribute progressively more than oldest ones. Thus, as the interaction number n grows (which means that such interaction is older), its value x_{jn}^{p} contributes less to the average mean. The weights follow a forgetting curve $w_n = e^{-n/S}$, based on an exponential function whose shape makes the weights decreasing more quickly as the evidence n is older (i.e., n is greater). S represents the relative Strength of the memory concept and it is used to customize the shape of the exponential function (the forgetting curve gets a more linear shape as S increases).

$$\overline{a}_j^p = \frac{\sum_{n=1}^{N} x_{jn}^{p} * e^{-n/S}}{\sum_{n=1}^{N} e^{-n/S}}$$

An expectation about a trust property is composed of two parameters, the average rating and the certainty. While the $average\ rating\ \overline{a} \in [0 \ldots 1]$ expresses the average outcome of the past interactions, i.e., the degree of past observations to support the truth of the new evidence, the $certainty\ c \in (0 \ldots 1]$ indicates the degree that the average rating is representative in the future. The higher the number of obtained evidences the more representative can be considered the calculated security expectation and trust value. Thus, $certainty\ c$ increases according to the number of collected historical evidences. The minimum level of certainty ($c = 0$) holds when there is no previous evidence whatsoever, while the maximum level ($c = 1$) is hold when the total number of interactions N reaches the number of expected number of evidence units. Therefore, the expectation of a certain property p for an IoT device j is defined as:

$$E_j^p = \overline{a}_j^p * c_j^p$$

To have an overall value for each dimension, the Trust Manager aggregates the expectation values for each singular property, and in the same category, following a bottom-up approach using a recursive function described in 1. As described in Sect. 5.1, the four dimensions are brokendown following a tree structure, having four root nodes corresponding to the four main dimensions and then split into categories until trust properties are reached. Each property and category has a weight assigned that can be customized by the user. Direct children trust properties in the same level are weighted and merged recursively in a single upper value assigned to the parent category, using a weighted mean with different configurable weights for each level. The Trust Manager goes through the tree structure aggregating values in categories until it reaches an overall value for each of the four dimensions. Notice that since the Reputation dimension only has one property, the dimension takes the value from the property and the recursive aggregation process is avoided. Algorithm 1 shows the pseudocode of the recursive function used to come up with a single trust value for a given dimension. Notice that each $Node$ in the algorithm represents one of the trust properties described previously. As can be seen, the algorithm performs a depth-first traversal, calculating a weighted mean at each parent node, until it finishes having the aggregated trust value of the root node.

Algorithm 1 Trust dimensions quantification. Recursive aggregation function

```
1: function RECURSIVE(node)
2:     sum = 0
3:     weightSum = 0
4:     childrenNodes[] = getChildren(node)
5:     if childrenNodes[] = ∅ then
6:         sum = node.value
7:     else
8:         for each node n in childrenNodes[] do
9:             v = RECURSIVE(n)
10:            sum = sum + v * n.weight
11:            weightSum = weightSum + n.weight
12:        end for
           sum = sum/weightSum
13:    end if
14:    return sum
15: end function
```

5.3 Fuzzy trust computation

To come up with a final crispy trust value about a given smart object j, the four overall values of each dimension are evaluated together using a fuzzy approach. Fuzzy logic suits perfectly to deal with the trust quantification since it manages effectively uncertain and aggregated subjective trust values. Fuzziness represents the degree of appropriateness of an element being considered as a member of a group. The fuzzy inference uses imprecise linguistic terms, like for instance *high security* or *low reputation*, to specify inference rules, which, when are evaluated in parallel, allow to come up with a overall crisp value of trust. The adoption of a fuzzy approach using linguistic variables to quantify trust makes it easier for users to describe their security and trust preferences. Humans think in vague qualities terms more than in quantitative ones and fuzziness provide a natural representation of the human knowledge about the relations and dependences between the variables involved. Furthermore, trust quantification based on fuzzy rules suits perfectly in the IoT world since the computation resources required by devices to run the fuzzy system are small.

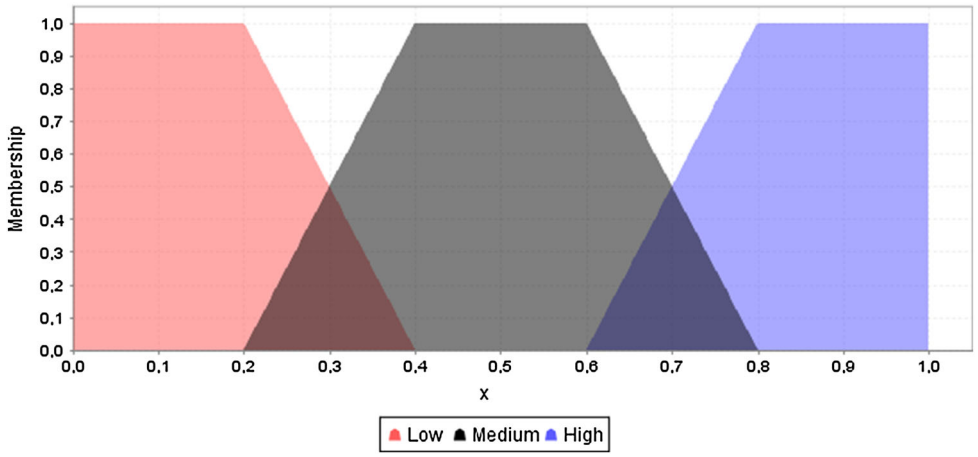

Fig. 2 Membership functions for the input variables definition

Fuzzy sets are used to define *terms* with respect of a variable used by the Fuzzy inference rules. The Fuzzy sets are based on the notion of a membership function $\mu(x)$ that describes the degree of a fuzzy variable x is member of a fuzzy set in such a way that a full membership is represented by 1, and no membership by 0. Thus, $\mu(x)$ maps x into the interval [0, 1]. The universe of discourse (UoD) of a fuzzy set represents the range of values that are considered. A fuzzy set S is usually represented $\mu_S(x) \in [0 \ldots 1]$.

The Trust Manager has a subcomponent that acts as Fuzzy Control System (FCS) that analyzes analog input values in terms of logical variables that take on continuous values between 0 and 1 and generates one output crispy value. A FCS is composed of four main parts: Fuzzification, Inference, Knowledge Base of Rules and Defuzzification. The FCS defines one output fuzzy variable and four fuzzy inputs variables according to the main four dimensions: Quality of Service, Reputation, Security and Social Relationship factor. Each of these input variables are linguistic variables which are mapped to sets of membership functions.

The four linguistic variables are defined in the same universe of discourse [0 . . . 1] with the same fuzzy sets. We have identified three fuzzy values *Low*, *Medium*, *High* to describe the four fuzzy variables. The three fuzzy values are defined by means of a trapezoidal membership function whose curve is a function that depends on four scalar parameters a, b, c, d: $f(x; a, b, c, d) = \max(\min(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}), 0)$. Figure 2 shows the graphical representation of the three fuzzy values for the four linguistic variables.

It is worth noting that membership functions have been chosen bearing in mind the computation limitations which characterize the Internet of Things scenarios. In this sense, trapezoidal functions are less complex (and therefore usually faster) than other common membership functions like Gaussians. Moreover, as can be seen the membership functions satisfy common fuzzy logic recommendations Yager and Filev (1994) like cardinality, normalization and completeness.

In addition, there is one fuzzy output linguistic variable called *Trustworthiness*, which defines four fuzzy values with different meaning. *Distrust*: the device will act against the best interests of another. *Untrust*: corresponds to the space between distrust and trust, in which a device is positively trusted, but perhaps not sufficiently to cooperate with it. *Trust*, it represents the range where the device ensures a minimum of reliability and acts as expected. *HighTrust* corresponds to the space where the evaluated entity can be confidently trusted. As can be seen in the graph of Fig. 3, the four membership functions are also defined using four trapezoidal functions.

The fuzzy control system uses the Singleton fuzzifier to convert a crisp input value to a fuzzy set of the input. Then, the inference process is performed with the set of fuzzy rules. The inference rules use the form *IF FuzzyInputs THEN FuzzyOutput* to generate a fuzzy output. These rules fuzzy outputs are afterwards aggregated by taking the fuzzy Max operator. To reasoning with the rules, the FCS is configured to use the Mandami Min implication operator. Thus, the Generalized Modus Ponens (GMP) clips the membership function of the consequent by the Degree of fulfillment (DOF). The following piece of code shows some fuzzy inference rules.

```
RULE 1: IF QualityService IS Low THEN trustworthiness IS untrust;
RULE 2: IF Security IS Medium THEN trustworthiness IS trust;
...
RULE n
```

Listing 1 Fuzzy Rules examples

Figure 4 shows the fuzzy grid which represents the fuzzy rules along with its associated weights. For the sake of clarity, the matrix only shows the minimal set of rules containing 12 rules with one antecedent and one consequent. Nonetheless, users can customize the set of adopted fuzzy rules and the weights, according to their security preferences, adding more complex rules with two or even

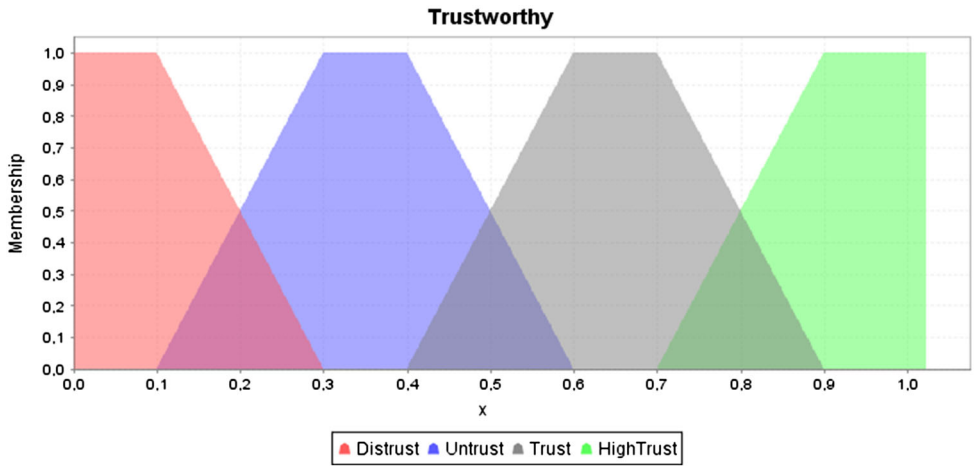

Fig. 3 Membership function for the output variable

	Low		Medium		High	
	Trust Output	w	Trust Output	w	Trust Output	w
QualityService	Untrust	1	Trust	1	HighTrust	1
Reputation	Distrust	0.8	Untrust	0.8	Trust	0.6
Security	Distrust	1	Trust	0.8	HighTrust	0.8
RelationshipFactor	Untrust	1	Trust	0.8	Trust	0.8

Fig. 4 Fuzzy rules matrix

three antecedents. Weights are established for each fuzzy rule allowing to define the degree of importance of each rule over the rest. The weights in the example matrix are established based on author's experience and preferences but they are intended to be customizable. Indeed, although it is not currently supported in our proposal, as future work we envisage to set up the weights adaptively using different mechanisms such as machine learning techniques.

As fuzzy inference example, Fig. 5 shows a graphical representation of the inference process taking into account only two rules *Rule1* and *Rule2* from Listing 1. The example takes as *QualityService* input value 0.25 and a value of 0.55 of *Security*. Each rule truncates the output function by the degree of fulfilment which in the first rule is 0.5 and in the second rule is 0.25. Then, the fuzzy outputs of each rule are aggregated together, as shown graphically in the bottom chart.

Finally, to come up with a final crisp value of trust, a defuzzification method over the resultant fuzzy output is applied. The trust modeluses the Center of Gravity (or centroid) method to obtain such a value. $x^* = \frac{\sum_{i=1}^{n} \mu(x_i) x_i}{\sum \mu(x_i)}$. In the above the example depicted in Fig. 5, taking both rules and supposing the above-mentioned inputs values, the defuzzied output and therefore the Trust Index, after applying the CoG formula would be 0.45.

6 Trust-based access control in IoT

The trust modelthat was previously described has been integrated into an access control mechanism, to enable a secure information exchange between trustworthy entities. This mechanism has been considered to be deployed on IoT scenarios where smart objects (e.g., smartphones, sensors, actuators, etc.) can maintain social relationships composing different kinds of bubbles (i.e., *Personal*, *Familial*, *Office* or *Community*). According to Fig. 6, each bubble is made up by a set of smart objects, along with an *Authorization Manager*, which is responsible for generating authorization credentials (e.g., it can be deployed on a user smartphone in the case of a personal bubble) for smart objects. Furthermore, each smart object has a Trust Manager, which is in charge of assessing the trustworthiness degree of an entity, according to the model presented in Sect. 5. In the case of IoT devices with tight resource constraints (i.e., class 1 devices), this entity is assumed to be deployed in a more powerful network component.

In the proposed trust-aware access control system, an *intra-bubble* communication happens when a smart object attempts to access another smart object that is part of the same bubble. Figure 6 shows the interactions at high level in the case of an *inter-bubble* communication between smart objects from different bubbles. Under this scenario, the purpose of the Trust Manager is twofold. On the one hand, it is used by the *requester* smart object to know the most trustworthy target among a set of devices providing the same service. On the other hand, it is employed by the *target* smart object to get the trust value associated with the requester under a specific transaction. This value is used, along with the authorization credential that is previously obtained from the *Authorization Manager*, to make the access control decision.

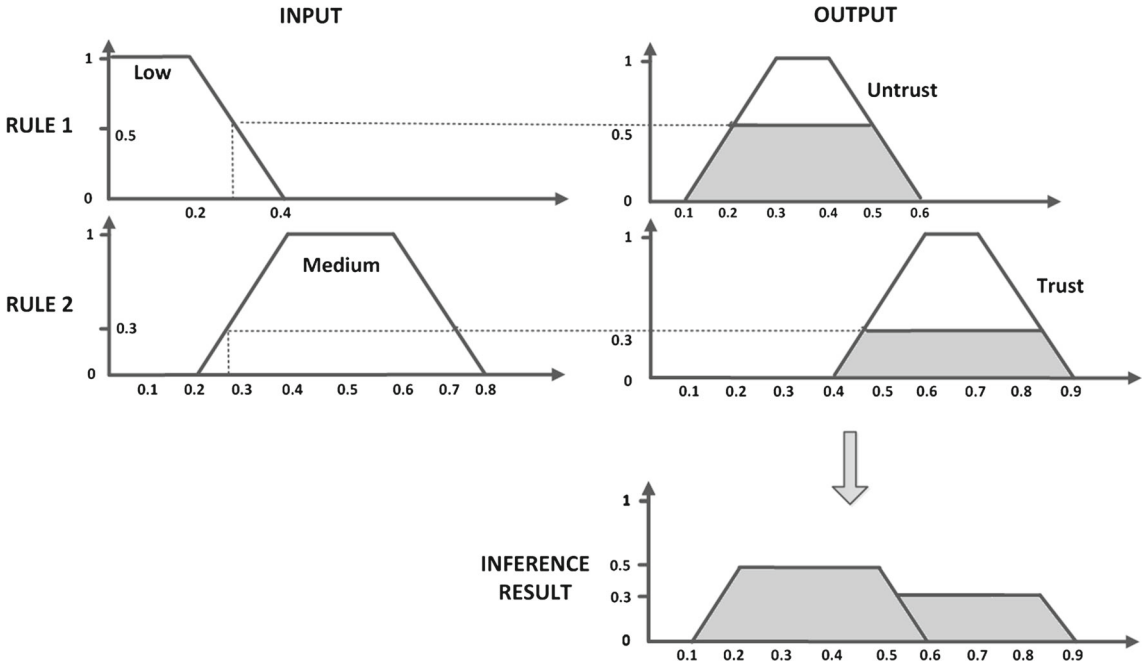

Fig. 5 Fuzzy rule graphical inference example

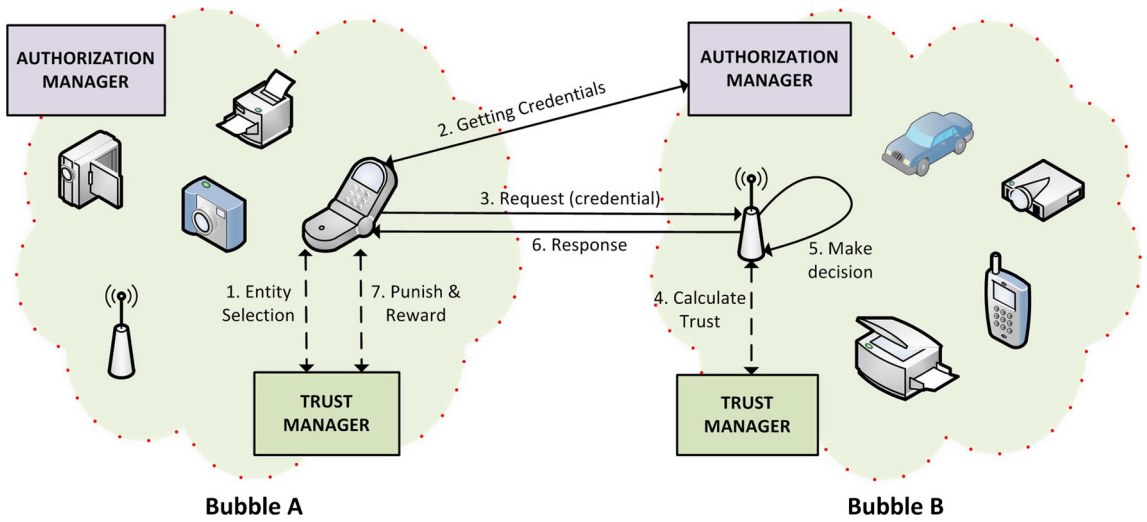

Fig. 6 TACIoT for IoT bubbles

Taking into account the main foundations of the proposed scenario, Fig. 7 shows the required message exchange for the realization of TACIoT. Before the detailed explanation of the messages, a brief description of the main involved components is provided:

- *Smart object* It is a device (e.g., a smartphone, printer, camera, sensor, etc.) that can act as both CoAP client and server offering services (e.g., temperature, location, etc.) in an IoT environment.
- *Trust manager* It is the component implementing the proposed trust model. In the case of a smart object with tight resource constraints (i.e., class 0 or class 1 device), it is a separate network element. In the case of more powerful smart objects (at least class 2 device), it is part of it.
- *Authorization manager* It is responsible for generating and sending authorization tokens to smart objects. In addition, it is composed of two subcomponents; the Policy Decision Point (PDP), which is in charge of making authorization decisions based on a XACML engine, and the Token Manager, which generates authorization credentials according to the authorization decisions.

The required message exchange for TACIoT has been split into four main stages, taking into account the generic steps of a trust and reputation system (Marti and Garcia-Molina

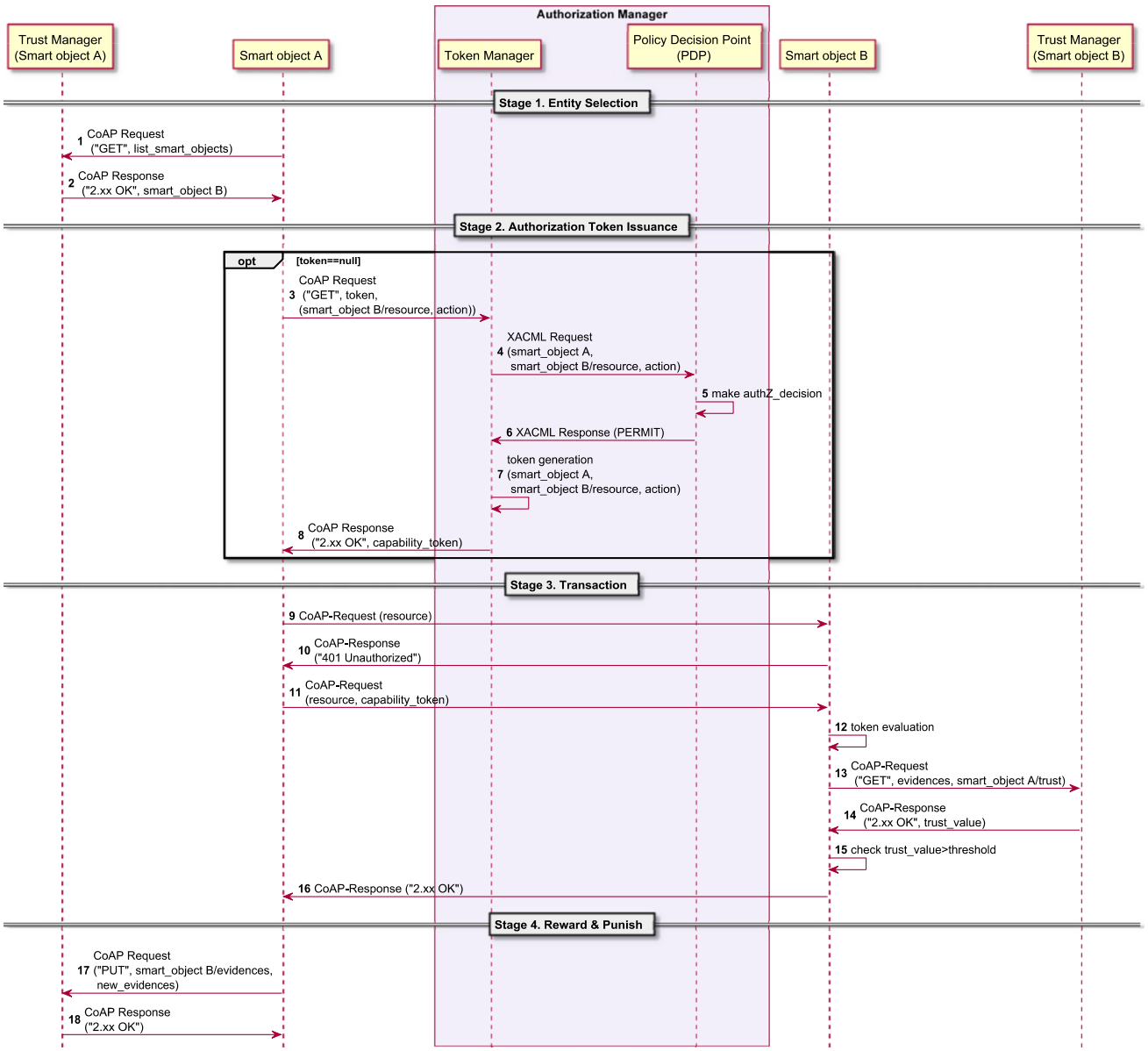

Fig. 7 Required interaction for the proposed scenario

2006). This scenario assumes a smart object A tries to get access a service that is provided by other smart object B in another bubble. During *Stage 1*, the smart object accesses the Trust Manager to know the most trustworthy smart object providing a specific service. For this purpose, it makes use of a CoAP request in which a list of smart objects (being identified by a URI) is attached as a new header list *SmartObjects* (in the case of a class 2 or more powerful device, this communication is performed within the smart object). At this point, it should be pointed out that it is assumed that the list of devices has been previously obtained through a service discovery mechanism (e.g., based on Jara et al. 2014). Upon receiving the request, the Trust Manager calculates the trust value associated with each of the smart objects within the list, according to the model presented in Sect. 5. Then, the Trust Manager sends a CoAP response to the smart object A with the URI of the most trustworthy smart object, in this case, the smart object B. At this stage, device A could even avoid requesting device B in case the trust value of B was not over a predefined threshold.

Before requesting access to device B, the smart object A needs a proper authorization credential (*Stage 2*). This stage is optional and it is carried out in case the device A has not got a valid capability token (e.g., it expired or was revoked). An authorization token example can be seen in listing 2. As can be seen, the access rights part 'ar' limits the access to the action *GET* over a resource *position*, as long as the trust threshold condition is over 0.7.

```
{"id": "Jd93_jZ8Ls5V0qP",
 "ii": 1412941013,
 "is": "coap://tokenManager.um.es",
 "su": "aB4wSICIXC1pm2pkW9YMPQyFudc=CPhYdgOAQwcOYgURwP1qO2WSv=",
 "de": "coap://smartObjectB.um.es",
 "si": "TqZaXuxZ5dmZU6k3PtiWwI3NrjH=7u5By5OHzlOOtq4TmkrZU2JPd=",
 "ar": [
  {"ac": "GET"
   "re": "position"
   "co": [{
     "t": 5,
     "u": trust,
     "v": 0.7}]}
 "nb": 1412941013,
 "na": 1412941456
}

Legend:
"id"-> identifier "ii"-> issued time "is"->issuer
"su"-> subject "de"-> device "si"-> signature "ar"-> accessRights
"ac"-> action "re"-> resource "co"-> condition "t"-> type
"u"-> unit "v"-> value "nb"-> not before "na"-> not after
```

Listing 2 Trust-aware capability Token example

In case the device A does not have a valid capability token it contacts the Token Manager entity, indicating the specific resource and action to be performed on the smart object B. Notice that this stage 2 is usually carried out just once to get the token, e.g., the first time device A tries to access to device B (or a bubble of device B). In this sense, the Token Manager sends a XACML request to the PDP to know if an authorization token must be granted or not to the smart object A. The Policy Decision Point evaluates the XACML policies using the policy engine and makes an authorization decision. In case of a PERMIT decision, the PDP generates an XACML obligation with a threshold trust value, to specify the risk associated with that access control decision. Then, the Token Manager generates a authorization credential or token and includes the threshold trust value as a new condition to be verified by the target device, i.e., the smart object B. Before delivering the authorization credential, the Token Manager signs the tokens.

Then, during the *Stage 3*, the smart object A uses the authorization credential for access to a service being hosted on the smart object B. This message exchange is based on our capability-based access control mechanism (DCapBac) (Hernández-Ramos et al. 2014). Basically, in this process, an authenticated key exchange is firstly carried to generate a session key. Then, such a session key is used to exchange securely the capability token that is used to get access to a specific resource. When device B receives the access request, it checks whether the token is valid or not checking the signature and if the token has expired. Then, as the access rights are contained in the token, the device B checks them against the requested action, including the conditions described in the token. Besides, since the device have the final say, it can check the current context, like for instance its battery level, before allowing device A to access to its resource.

In addition, once the capability token is evaluated by the smart object B, the access control process at device B takes into account the trust value associated with the requester device, in this case the smart object A. This trust value is based on previous evidences which are stored by the Trust Manager of B, as well as a set of evidences associated with the current transaction. Specifically, in the case of an inter-bubble communication, the *Interest-In-Common* and *Friends-In-Common* trust properties are considered for the trust calculation as explained in Sect. 5.1. Once the smart object B receives the trust value, it verifies this number is greater than the *threshold* trust value, which is contained in the capability token. If that condition is fulfilled, the request is accepted and the service is provided to the smart object A.

Finally, during the *Stage 4*, depending on the satisfaction degree of the smart object A regarding the transaction, a set of evidences about the interaction is sent to its Trust ManagerA in order to update the trust value associated with the smart object B. These evidences are, in fact, values of the trust properties defined in Sect. 5.1. At least the *SucessfullInteraction* property of the *Quality of Service* dimension of the trust model should be assessed during the reward stage. Then, the Trust Manager of A in the event of new incoming evidences computes again the trustworthy of the assessed device B to have the new trust score ready for future requests coming in the other way around, i.e., from device B.

7 Evaluation results

7.1 Implementation and testbed features

The trust model of Sect. 5.1 has been implemented as part of the Trust Manager component of the security framework. The Trust Manager implementation is able to quantify trust property expectations of devices based on historical evidences, calculate trust dimensions values from the expectations and compute trustworthiness based on such trust dimensions. The Trust Manager has been implemented in Android SDK and tested on Android Platform 2.3.3 (API level 10) to cover a wide variety of Android devices with moderate hardware capabilities. Nonetheless, it also works properly in actual platforms like Android 4.4. It features an interface to get trust values about a device or set of devices as well as an interface to feed the Trust Manager with new evidences about another device (reward–punish operation). It uses CoAP as application layer protocol to enable constrained devices to get the trust values and put new evidences. To deal with the fuzzy trust quantification, the Trust Manager implementation relies on the jfuzzylite library (Rada-Vilela 2014), which is a lightweight and open-source fuzzy logic control library for Android.

The proposed trust-aware access control mechanism was validated on a real testbed, in which the main components of TACIoT were instantiated. According to the mechanism, three main entities are required to carry out the main function-

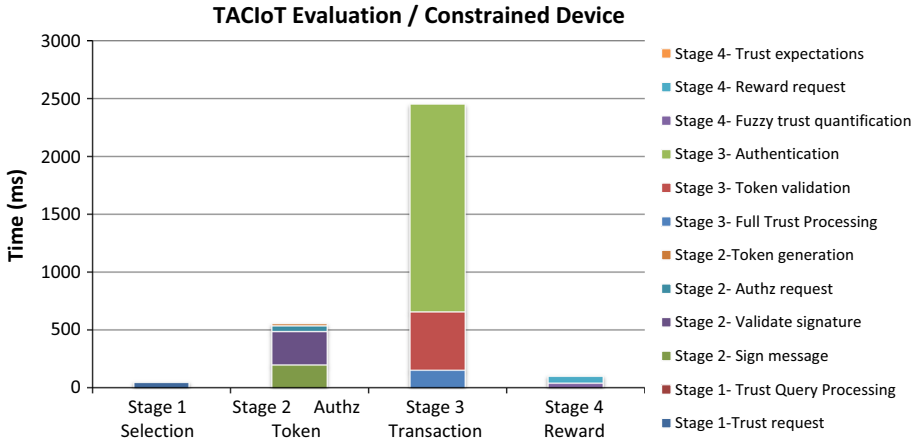

Fig. 8 TACIoT evaluation—constrained devices

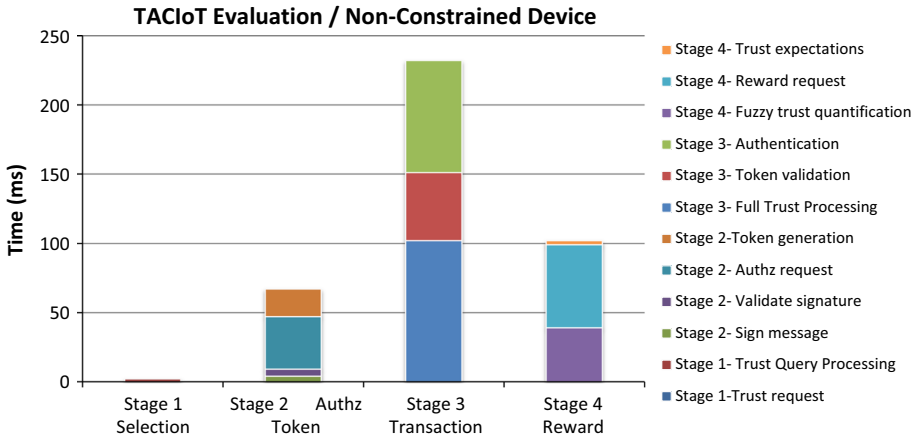

Fig. 9 TACIoT evaluation—non-constrained devices

ality of the proposed approach. First, the Trust Manager was deployed in a smartphone with ARM Cortex A8 Processor 1Ghz and 512 MiB RAM. Furthermore, the Authorization Manager was developed on a common computer with an Intel Core i5 processor with 2.27 GHz and 4GB RAM. Moreover, given the heterogeneous nature of IoT scenarios, the functionality of the smart objects was implemented in two different hardware components. On the one hand, two resource-constrained smart objects were implemented on a JN5148 mote equipped with Contiki OS. It provides a low-power 32-bit load and stores RISC with a programmable CPU speed which can be set to 4, 8, 16 or 32 MHz. Furthermore, it has a unified memory architecture with 128 kbytes of ROM, 128 kbytes of RAM and a 32-byte One Time Programmable (OTP) eFuse memory. On the other hand, two more powerful smart objects were considered and developed on the same hardware as the Trust Manager.

7.2 TACIoT validation

To demonstrate the feasibility of TACIoT, we executed different tests of the main stages of the scenario proposed in Sect. 6. As mentioned above, these tests were performed considering two scenarios according to the hardware features of the smart objects. Figure 8 recaps the performance times for each of the main tasks required at each of the four main stages in constrained devices. Similarly Fig. 9 shows the achieved times for the non-constrained devices. For a better understanding of the graphs, series are grouped in a different bar for each stage and arranged in a top-down order, following the order appeared in the legend of the graphs. Notice that the vertical axis has a different time scale in both charts since the non-constrained devices require more time to accomplish the different operations.

During the Stage 1 of TACIoT, the following tasks are required:

- *Trust request* This includes the time required for CoAP messages processing. This delay is only necessary in constrained devices in which the Trust Manager cannot be deployed in the same device.
- *Trust query processing* This includes the delay that is required by the Trust Manager to get a trust value about a device. The Trust manager computes trust only after adding new evidences which is not the case in this operation.

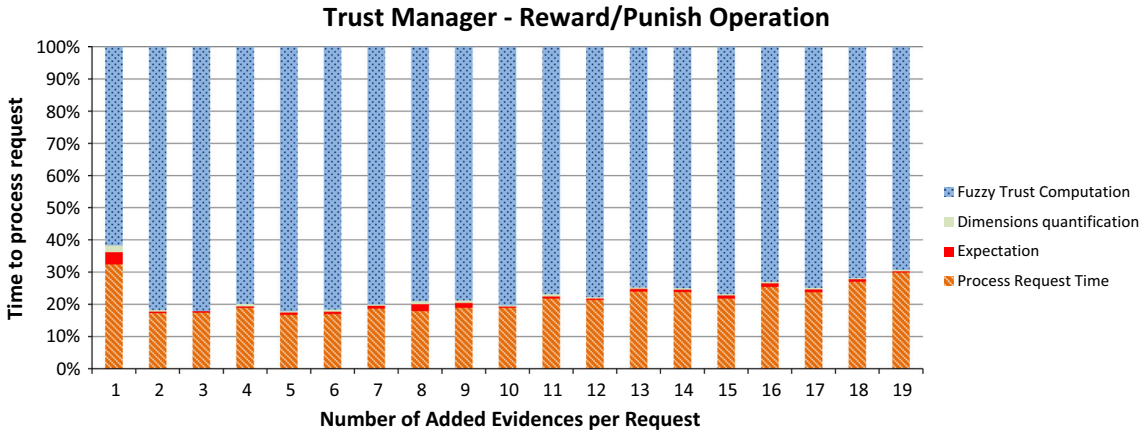

Fig. 10 Trust quantification, reward operation

As can be seen from the chart of Fig. 8, the stage 1 is the fastest compared to the other three since it is not necessary to compute trust and the Trust Manager just returns the actual trust value of the requested device.

During Stage 2, the requester smart object obtains an authorization credential to access a service on the target smart object. It includes:

- *Sign message* The time required for an authenticated CoAP message exchange between the smart object and the Token Manager.
- *Validate signature* To validate the signature of the response. This operation strongly differs according to the kind of device validating the signature, since in the constrained device the Elliptic Curve Digital Signature (ECDSA) takes around 288ms whereas in android with Android SDK it takes around 5 ms.
- *Authz request* The time required to process the CoAP messages.
- *Token generation* It includes the token generation in the Token Manager. It should be pointed out that the PDP time is omitted since it is strongly dependent on the policy engine, the use case and the specific representation of XACML policies.

The transaction between both smart objects is carried out at the Stage 3, including:

- *Authentication* It refers to the time for ECDSA operations as well as the delay required to generate a session key, to protect the CoAP request in which the token is attached in the initial stage. Again this time is very small in non-constrained devices.
- *Trust processing* The time to obtain a trust value from the Trust Manager about a device, attaching in the request new evidences with information of the current transaction. It includes the time to infer a new trust value based on the new attached evidences. In the case of a constrained smart object without Trust Manager installed in it, it also includes the time needed to perform the CoAP request to the Trust Manager.
- *Token validation* It includes the delay to evaluate the authorization credential and the time to validate the Token Manager's signature. Again this time strongly differs in both kind of devices.

As can be seen in the charts of Fig. 8, Stage 3 is the heaviest one in both cases, constrained and non-constrained devices. This is due to the fact that this stage requires more cryptographic operations, which are more expensive compared to trust quantification.

Finally, during Stage 4 the device provides information about the transaction just carried out, for the Trust Manager to be up-to-date about the device B behavior. Stage 4 can be split into three main operations.

- *Reward request* This time represents the delay required to process CoAP request including processing the evidences coming in the request.
- *Trust expectations* This time represents the time required to quantify expectation values for each trust property defined in 5.1 based on historical evidences and the new ones, as well as generates the current four dimension values.
- *Fuzzy trust quantification* This time is the heaviest one during the trust computation and refers to the time needed to infer the result by the fuzzy control system.

As can be seen in Fig. 8, the time required to accomplish stage 4 is equivalent to carry out the Trust processing task of stage 3.

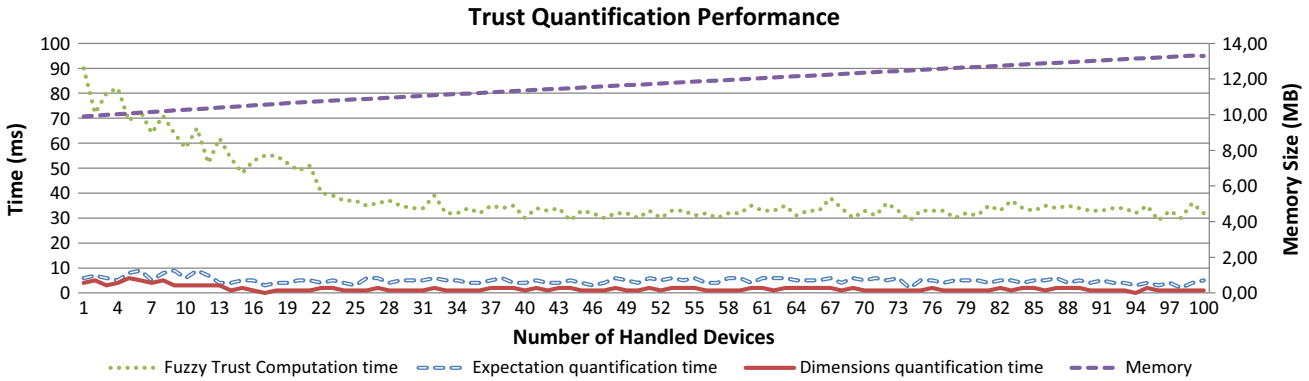

Fig. 11 Trust quantification performance

It is worth analyzing the behavior of the Trust Manager when it has to handle different amount of evidences attached in a reward–punish request. As can be seen in Fig. 10, it takes barely the same time to quantify trust after performing the reward operation, regardless of whether the request is attaching one evidence or the full set of evidences (the evidences of the 19 trust properties identified in Sect. 5.1). The small difference lies in the time to decode a request with more evidences attached.

To analyze the Trust Manager performance, we measured the time that is required to compute trustworthiness when it has to deal with different amount of devices. To see the behavior trend, we considered up to 100 devices. The test generates randomly evidences for each of the 19 trust property values and queries the Trust Manager attaching all of them each time. By default, for each device the Trust Manager is configured to hold up to 10 historical evidences per trust property, i.e., it holds up to 190 evidences per device, then it dismisses the oldest ones.

Figure 11 shows the trust quantification performance. As it was predictable, as the amount of handled devices is higher, the Trust Manager requires more memory to handle all the historical evidences. Nonetheless, the memory consumption follows a moderate plain linear trend. Moreover, after some executions, the time to compute trust remains more or less steady regardless of the amount of devices handled, since the information about other devices does not influence the trust quantification.

The achieved results show reasonable performance times as well as the suitability of TACIoT for IoT scenarios, whereby heterogeneous devices can interact each other in a trusted and reliable way.

8 Conclusions

As part of the challenging goal of designing and implementing a ARM-compliant security framework for IoT, this paper provided a trust-aware access control system (TACIoT) which takes into account devices' trust values to make authorization decisions accordingly. The paper provided a trust model especially meant for IoT which takes into account, four dimensions, i.e., quality of service, reputation, security aspects and social relationships, to compute trust values about IoT devices. The trust model was implemented as part of a Trust Manager of our IoT security framework making use of a fuzzy control system, which relies on the four trust dimensions, which in turn, are quantified from historical trust property evidences.

The feasibility of TACIoT was demonstrated by testing the software implementation on real scenarios for constrained and non-constrained devices. The achieved results show reasonable times when performing the trust-aware access control process for both kinds of devices.

As future work, we envisage to continue with further experiments in order to evaluate the accuracy of the trust quantification model in real IoT scenarios. We also expect to continue deploying the security components of the IoT security framework, such as the context-aware Identity Management system and the Group Manager component. The challenging goal is to provide complete secure and privacy-preserving environment where IoT devices can interact each other in a trusted way and share data securely within bubbles and communities.

Acknowledgments This work has been sponsored by European Commission through the FP7-SOCIOTAL-609112 EU Projects, and the Spanish Seneca Foundation by means of the Excellence Researching Group Program (04552/GERM/06).

References

A socially aware citizen-centric Internet of Things C (2013) Eu fp7 sociotal project. http://sociotal.eu

Architecture D.I.S. proof of concept I.P.B. Eu fp7 butler project (2013)

Atzori L, Iera A, Morabito G (2010) The internet of things: a survey. Elsevier Comput Netw 54(15):2787–2805

Atzori L, Iera A, Morabito G, Nitti M (2012) The social internet of things (siot)-when social networks meet the internet of things:

concept, architecture and network characterization. Comput Netw 56(16):3594–3608
Bao F, Chen IR, Guo J (2013) Scalable, adaptive and survivable trust management for community of interest based internet of things systems. In: Autonomous Decentralized Systems (ISADS), 2013 IEEE Eleventh International Symposium on, pp 1–7. IEEE
Bao F, Chen IR (2012) Dynamic trust management for internet of things applications. In: Proceedings of the 2012 international workshop on Self-aware internet of things, pp 1–6. ACM
Bassi A, Bauer M, Fiedler M, Kramp T, van Kranenburg R, Lange S, Meissner S (2013) Enabling things to talk. Springer, Berlin, Heidelberg
Bernabe BJ, Luis Hernndez MVM, Skarmeta A (2014) Privacy-preserving security framework for a social-aware internet of things. In: UCAm I 2014, pp 408–415
Chen D, Chang G, Sun D, Li J, Jia J, Wang X (2011) Trm-iot: a trust management model based on fuzzy reputation for internet of things. Comput Sci Inf Syst 8(4):1207–1228
Chen D, Chang G, Sun D, Jia J, Wang X (2012) Modeling access control for cyber-physical systems using reputation. Comput Electr Eng 38(5):1088–1101
Crockford D (2006) RFC 4627: The application/json Media Type for Javascript Object Notation (JSON). IETF RFC 4627. http://www.ietf.org/rfc/rfc4627.txt
Ferraiolo D, Cugini J, Kuhn R (1995) Role-based access control (RBAC): features and motivations. In: Proceedings of 11th Annual Computer Security Application Conference, pp 241–48
Gerdes S (2014) Actors in the ace architecture. IETF Internet Draft, draft-gerdes-ace-actors-01
Gusmeroli S, Piccione S, Rotondi D (2013) A capability-based security approach to manage access control in the internet of things. Math Comput Model 58(5–6):1189–1205
Heer T, Garcia-Morchon O, Hummen R, Keoh SL, Kumar SS, Wehrle K (2011) Security challenges in the ip-based internet of things. Wirel Pers Commun 61(3):527–542
Hernández-Ramos JL, Jara AJ, Marín L, Skarmeta AF (2014) Dcapbac: Embedding authorization logic into smart things through ecc optimizations. Int J Comput Math 1–22. doi:10.1080/00207160.2014.915316
Jara AJ, Lopez P, Fernandez D, Castillo JF, Zamora MA, Skarmeta AF (2014) Mobile digcovery: discovering and interacting with the world through the internet of things. Pers Ubiquitous Comput 18(2):323–338
Langheinrich M (2001) Privacy by designprinciples of privacy-aware ubiquitous systems. In: Ubicomp 2001: Ubiquitous Computing, pp 273–291. Springer
Mahalle PN, Thakre PA, Prasad NR, Prasad R (2013) A fuzzy approach to trust based access control in internet of things. In: Wireless Communications, Vehicular Technology, Information Theory and Aerospace & Electronic Systems (VITAE), 2013 3rd International Conference on, pp 1–5. IEEE
Mahalle, PN, Anggorojati B, Prasad NR, Prasad R (2012) Identity driven capability based access control (ICAC) for the Internet of Things. In: Proceedings of the 6th IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS), Bangalore, India, pp 49–54. IEEE
Marin L, Jara A, Skarmeta A (2013) Shifting primes on openrisc processors with hardware multiplier. In: Information and Communication Technology, pp 540–549. Springer
Marti S, Garcia-Molina H (2006) Taxonomy of trust: categorizing p2p reputation systems. Comput Netw 50(4):472–484
Medaglia CM, Serbanati A (2010) An overview of privacy and security issues in the internet of things. In: The Internet of Things, pp. 389–395. Springer
Nitti M, Girau R, Atzori L (2013) Trustworthiness management in the social internet of things. IEEE Trans Knowl Data Eng 26(5):1253–1266
Rada-Vilela J (2014) Fuzzylite: a fuzzy logic control library. http://www.fuzzylite.com
Rescola E, Modadugu N (2006) Rfc 4347: Datagram transport layer security (dtls). Request for Comments, IETF
Rissanen E (2012) extensible access control markup language (xacml) version 3.0 oasis standard
Saied Ben, Olivereau Y, Zeghlache D, Laurent M (2013) Trust management system design for the internet of things: a context-aware and multi-service approach. Comput Secur 39:351–365
Schaffers H, Komninos N, Pallot M, Trousse B, Nilsson M, Oliveira A (2011) Smart cities and the future internet: towards cooperation frameworks for open innovation. Springer
Seitz L, Selander G (2014) Problem description for authorization in constrained environments. IETF Internet Draft, draft-seitz-ace-problem-description-01
Shelby Z, Hartke K, Bormann C (2014) The constrained application protocol (coap). IETF RFC 7252:10
Weiser M (1991) The computer for the 21st century. Sci Am 265(3):94–104
Yager RR, Filev D (1994) Essentials of fuzzy modeling and control. Wiley, New York
Yuan E, Tong J (2005) Attributed based access control (ABAC) for web services. In: Proceedings of the 12th IEEE International Conference on Web Services (ICWS), Orlando, USA. IEEE
Ziegler S, Crettaz C, Ladid L, Krco S, Pokric B, Skarmeta AF, Jara A, Kastner W, Jung M (2013) Iot6-moving to an ipv6-based future iot. Springer, Berlin, Heidelberg